```
      .attr("class", "sample")
      .on("mouseover", function() {
        d3.select(this).style("fill", "pink");
      });

  zoomFinished();
```

We have to call zoomFinished (listing 11.9) to draw the canvas countries with SVG triangles.

This allows us to restore our canvas drawing code for triangles to the `zoomed` function and to move the SVG rendering code out of the `zoomed` function and into a new `zoomFinished` function. We also need to hide the SVG triangles when zooming or panning starts by creating a `zoomInitialized` function that itself also fires the `zoomed` function (to draw the triangles we just hid, but in canvas). Finally, our `zoomFinished` function also contains the canvas drawing code necessary to only draw the countries. The different drawing strategies based on `zoom` events are shown in table 11.1.

Table 11.1 Rendering action based on `zoom` event

`zoom` event	*Countries* rendered as	Triangles rendered as
`zoomed`	Canvas	Canvas
`zoomInitialized`	Canvas	Hide SVG
`zoomFinished`	Canvas	SVG

As you can see in the following listing, this code is inefficient, but I wanted to be explicit about this functionality, because it's a bit convoluted.

Listing 11.9 Zoom functions for mixed rendering

```
function zoomed() {
  projection.translate(mapZoom.translate()).scale(mapZoom.scale());

  var context = d3.select("canvas").node().getContext("2d");
  context.clearRect(0,0,500,500);
  canvasPath.context(context);

  context.strokeStyle = "black";
  context.fillStyle = "gray";
  context.lineWidth = "1px";
  for (var x in countries.features) {
    context.beginPath();
    canvasPath(countries.features[x]);
    context.stroke()
    context.fill();
  }

  context.strokeStyle = "black";
  context.fillStyle = "rgba(255,0,0,.2)";
  context.lineWidth = 1;
  for (var x in sampleData) {
    context.beginPath();
```

Draws all elements as canvas during zooming